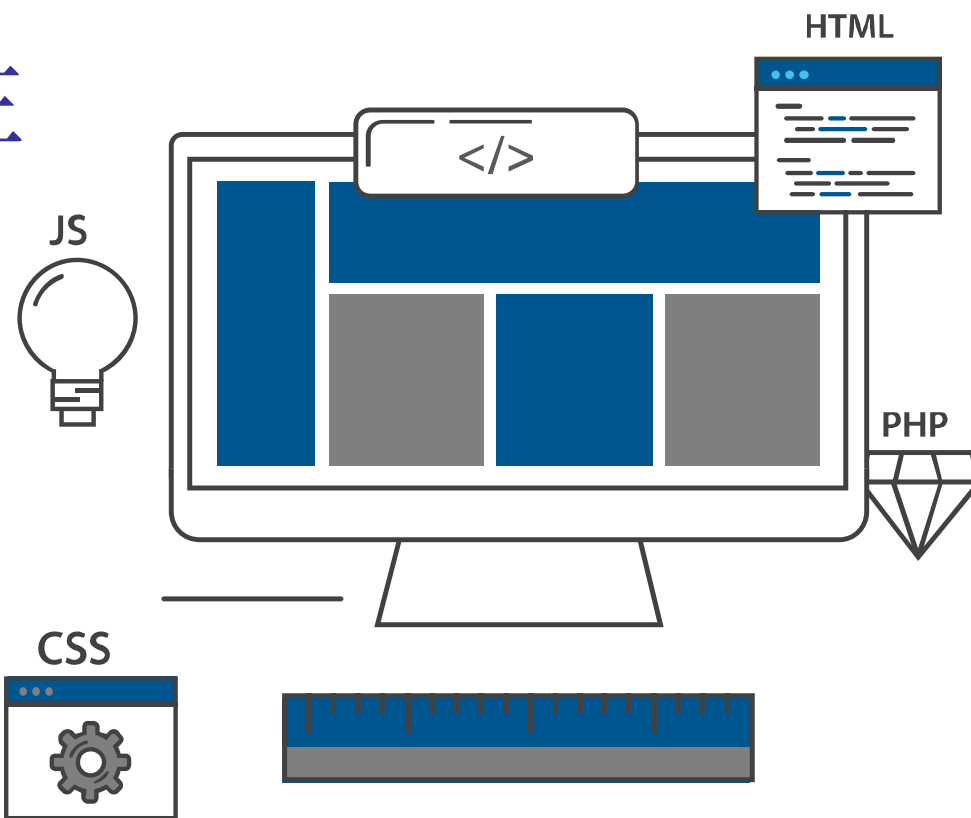
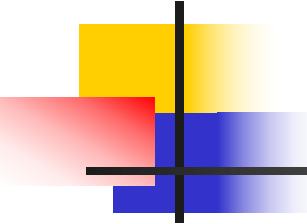


第八章 NP与计算的难解性

李翔

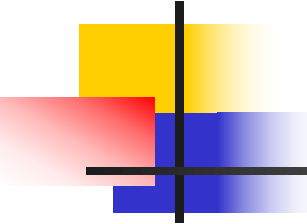
<https://implus.github.io/>





内容

- ✓ 多项式时间归约
- ✓ 使用“零件”归约: SAT
- ✓ 有效证书和NP的定义
- ✓ NP完全问题
- ✓ 排序、划分、图着色、数值问题
- ✓ 难问题的部分分类



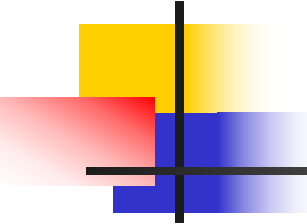
NP与计算的难解性

- 贪心算法 区间调度: $O(n \log n)$.
- 分治策略 FFT $O(n \log n)$.
- 动态规划 编辑距离 $O(n^2)$

最开始我们把**多项式时间**作为效率的工作概念

面对一些困难的问题，我们即不知道这些问题存在多项式时间算法，也不能证明问题不存在多项式时间算法

这里将会对“**困难**”问题提出一个清晰的概念：在计算上实际上是难的，虽然我们 cannot 证明它---NP完全

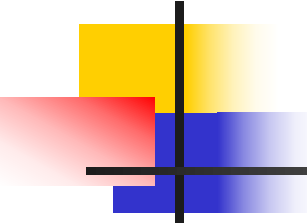


多项式时间归约

- 如果对于问题Y:
 - 能够用多项式个标准的计算步骤;
 - 加上多项式次调用解问题X的黑盒子来解问题Y

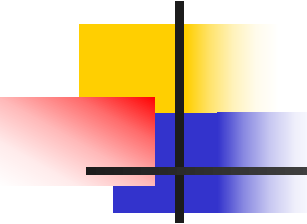
那么记作 $Y \leq_p X$;

读作 “**Y多项式时间可归约到X**”;或 “X至少像Y一样的难(相对于多项式时间)”



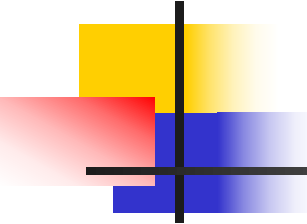
多项式时间归约

- 定理8.1 假设 $Y \leq_p X$ ，如果 X 能在多项式时间内求解，则 Y 也能在多项式时间内求解。
- 定理8.2 假设 $Y \leq_p X$ ，如果 Y 不能在多项式时间内解决，则 X 不能在多项式时间内解决。



多项式时间归约

- 归约的基本策略
 - 简单等价归约.
 - 从特殊情形归约.
 - 使用零件归约.

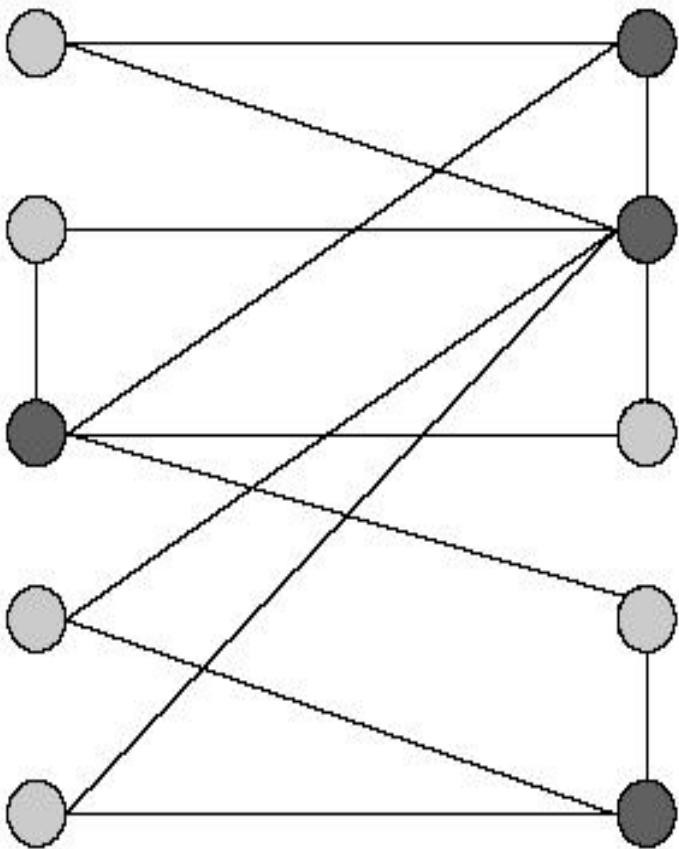


多项式时间归约

独立集

- 在图 $G=(V,E)$ 中, 如果顶点集合 $S \subseteq V$ 中的任意两点之间没有边, 则称 S 是**独立的**。
- **独立集问题:**
- 给定图 G 和数 k , 问 G 包含大小至少为 k 的独立集吗?

多项式时间归约



存在大小至少为**6**的独立集吗?

Yes

存在大小至少为**7**的独立集吗?

No

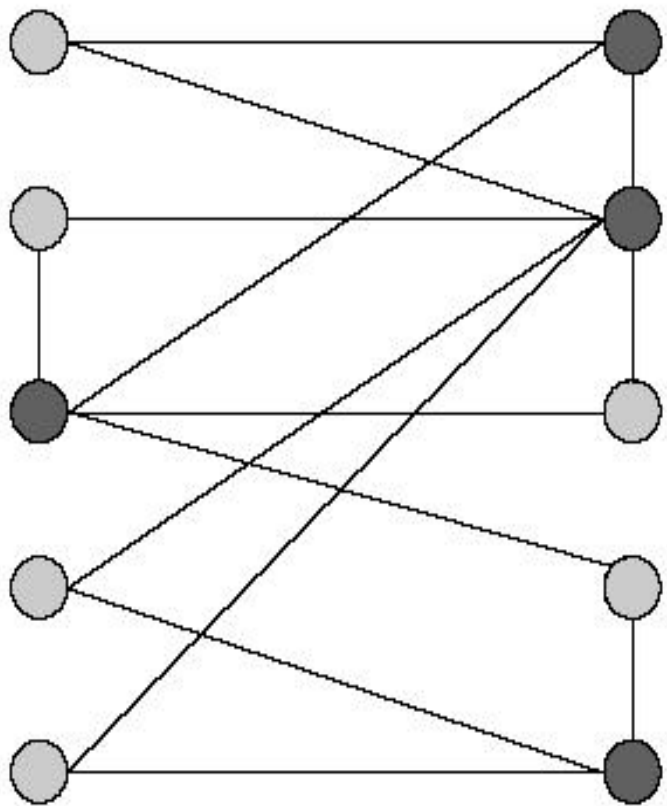


多项式时间归约

现在考虑另外一个基本图论问题：顶点覆盖

- 给定图 $G=(V,E)$,如果每一条边 $e \in E$ 至少有一个端点在 S 中, 则称 S 是一个**顶点覆盖**。
- **顶点覆盖**问题:
- 给定图 G 和数 k ,问 G 是否包含大小至多为 k 的顶点覆盖?

多项式时间归约



存在大小至多为**4**的顶点覆盖吗?

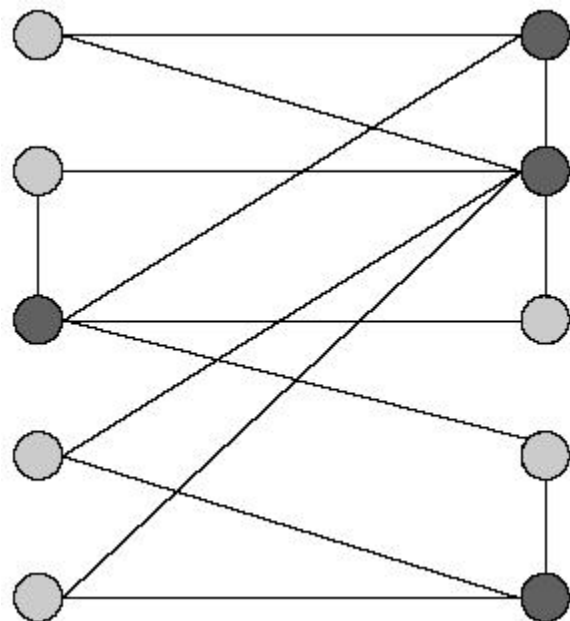
Yes

存在大小至多为**3**的顶点覆盖吗?

No

多项式时间归约

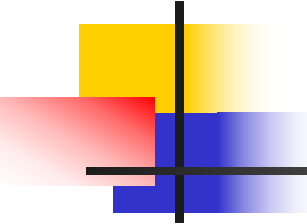
- 看起来顶点覆盖问题，独立集问题可能存在一定联系



独立集



顶点覆盖



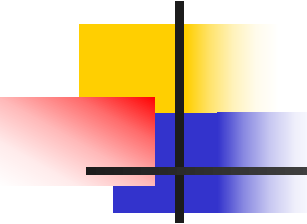
多项式时间归约

- 命题.
- $\text{VERTEX-COVER} \equiv_p \text{INDEPENDENT-SET}$
- 下面将介绍从特殊情形归约到一般情形的方法



多项式时间归约

- 集合覆盖：试图用一组较小的集合覆盖一个任意的对象集合
- 应用实例
 - m 个可供利用的软件；
 - 集合 U 由整个系统必须具备的 n 个性能组成；
 - 第 i 个软件包含性能集合 $S_i \subseteq U$.
 - 目标：用尽可能少数的软件包含所需具备的 n 个性能。



多项式时间归约

- 集合覆盖问题
- 给定 n 个元素的集合 U , U 的子集 $S_1, S_2, \dots, S_m$ 以及数 k , 是否存在数目至多为 k 的子集合, 其并集等于 U

例子

$U = \{1, 2, 3, 4, 5, 6, 7\}$

$k = 2$

$S_1 = \{3, 7\}$

$S_4 = \{2, 4\}$

$S_2 = \{3, 4, 5, 6\}$

$S_5 = \{5\}$

$S_3 = \{1\}$

$S_6 = \{1, 2, 6, 7\}$



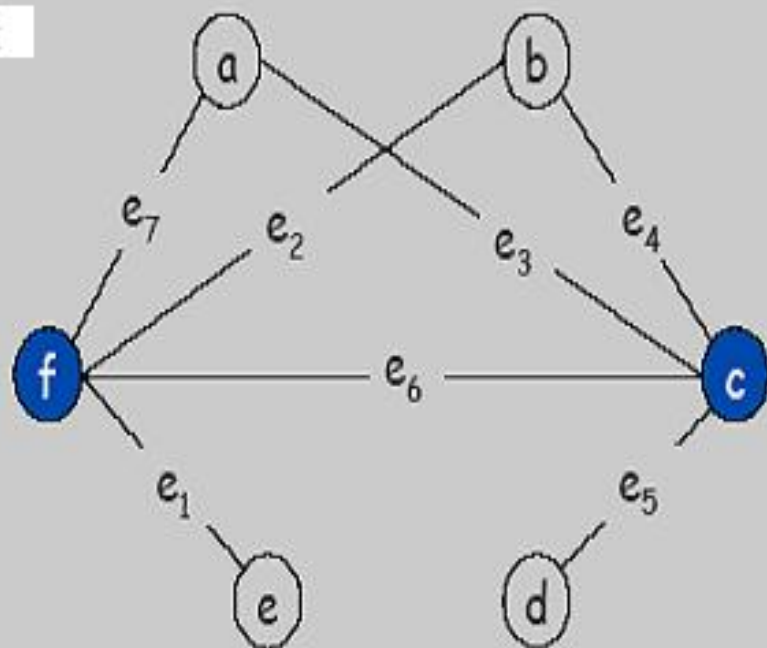
多项式时间归约

直觉上告诉我们，集合覆盖比顶点覆盖更具一般性

- 定理8.6 顶点覆盖 $\leq_p$ 集合覆盖
- 证明：我们构造集合覆盖的一个实例， $U=E$. 在顶点覆盖中，每次取一个顶点覆盖所有与它关联的边。于是，对每一个顶点 $i \in V$, $S_i = \{e \in E : e \text{ 连接到 } i\}$;
 $k = k$

多项式时间归约

顶点覆盖



集合覆盖

$U = \{1, 2, 3, 4, 5, 6, 7\}$

$k = 2$

$S_a = \{3, 7\}$

$S_b = \{2, 4\}$

$S_c = \{3, 4, 5, 6\}$

$S_d = \{5\}$

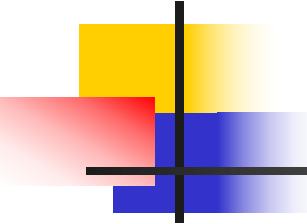
$S_e = \{1\}$

$S_f = \{1, 2, 6, 7\}$



多项式时间归约

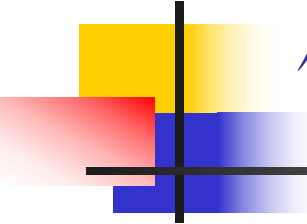
- 注意到集合覆盖是顶点覆盖的自然推广
- 类似的，独立集的自然推广是关于集合的包装问题
- **集合包装**问题：希望把大量集合包装在一起，限制他们中的任意两个都不重叠。
- 例子：设想有 n 个不能共享的资源集合 U , m 个软件进程，第 i 个进程运行需要的资源集为 S_i , 需要从这些进程中寻找一组进程，使得他们能够同时运行，而且所需资源没有重叠冲突。



多项式时间归约

- 问题描述:
- 给定 n 个元素的集合 U , U 的子集 $S_1, \dots, S_m$
以及数 k , 问在这些子集中至少有 k 个两两不相交吗?

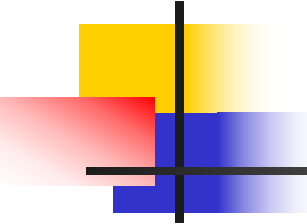
定理8.7 独立集 $\leq_p$ 集合包装



使用“零件”归约

- 给定n个布尔变量 $x_1, \dots, x_n$ 的集合X, 每个布尔变量可以取值0,1
- 项(Literal): 一个布尔变量或者其逆 x_i or $\overline{x_i}$
- 子句(Clause): 项的析取 $C_j = x_1 \vee \overline{x_2} \vee x_3$
- 合取范式(Conjunctive Normal Form, CNF): 一个命题公式 Φ 由子句的合取范式构成。

$$\Phi = C_1 \wedge C_2 \wedge C_3 \wedge C_4$$

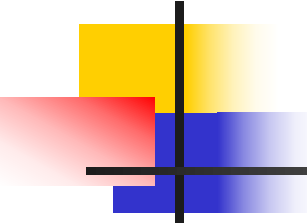


使用“零件”归约

- 如果赋值 v 满足每一个子句 $C_1, \dots, C_k$, 也就是合取式 $\phi = c_1 \wedge \dots \wedge c_k$ 值为1, 称 v 是关于 $C_1, \dots, C_k$ 的一个满足的赋值, 又称这组子句是可满足的。

例子: $(\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_2 \vee x_3) \wedge (\bar{x}_1 \vee \bar{x}_2 \vee \bar{x}_3)$
是否可满足?

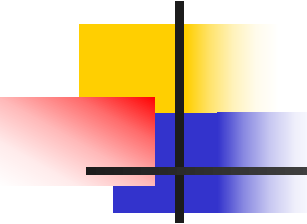
Yes: $x_1 = 1(\text{true}), x_2 = 1(\text{true}), x_3 = 0(\text{false}).$



使用“零件”归约

- 可满足性问题: (SAT)
- 给定变量集合 $X = \{x_1, \dots, x_n\}$ 上的一组子句 $C_1, \dots, C_k$, 或者是对于 $\phi = c_1 \wedge \dots \wedge c_k$ (CNF), 问存在满足的真值赋值吗?

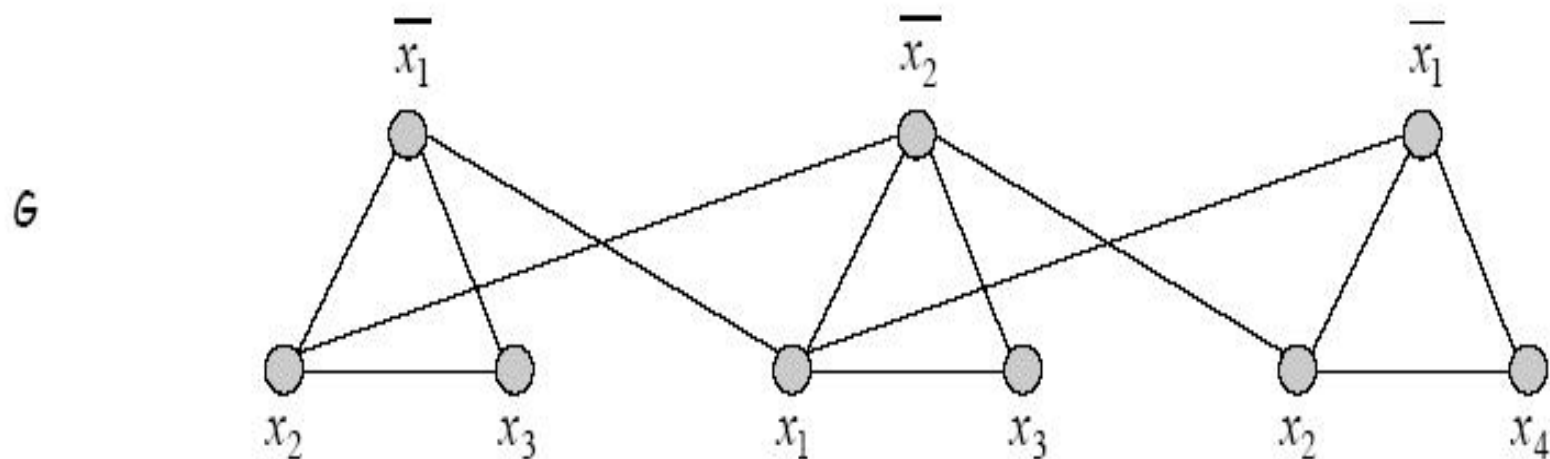
3-SAT:三元可满足性:
要求每一个子句的长为3



使用“零件”归约

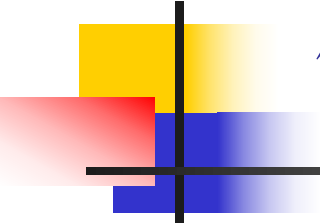
- 可满足性和三元可满足性是基本的组合搜索问题，以非常梗概的方式包含一个难计算问题的基本要素，必须作出 n 个独立的决定满足一组约束。
- 定理8.8 $3\text{-SAT} \leq_p \text{独立集}$.

使用“零件”归约



$k=3$

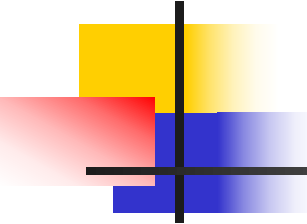
$$\Phi = (\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (\bar{x}_1 \vee x_2 \vee x_4)$$



使用“零件”归约

- 规约的传递性
- 定理8.9 如果 $X \leq_p Y$ 且 $Y \leq_p Z$, 则 $X \leq_p Z$.

证明：把两个算法组合起来

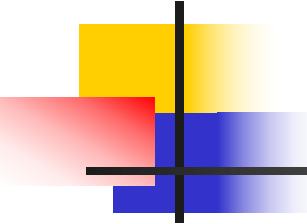


使用“零件”归约

- 简单等价归约: $\text{独立集} \equiv_p \text{顶点覆盖}$.
- 从特殊情形归约: $\text{顶点覆盖} \leq_p \text{集合覆盖}$.
- 使用“零件”归约: $3\text{-SAT} \leq_p \text{独立集}$

于是对于这几个问题的难度我们有了一个比较关系:

$3\text{-SAT} \leq_p \text{独立集} \leq_p \text{顶点覆盖} \leq_p \text{集合覆盖}$.



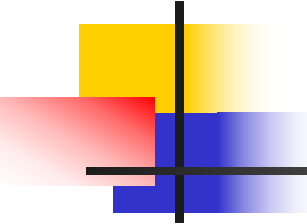
8.3 有效证书和NP的定义

- 一个计算问题的输入被编码成有穷的二进制串 s ,串 s 的长度记为 $|s|$.把判定问题 X 等同于由对它的答案为“**Yes**”的串组成的集合
- 判定问题的算法 A 接受输入串 s 并返回值“yes”或“no”,返回的值记为 $A(s)$,如果对所有的串 s , $A(s)=yes$ 当且仅当 $s \in X$,则称 **A 解决问题 X** .



NP的定义

- 如果存在多项式 $p(\cdot)$ 使得对每一个输入串 s , 算法 A 对 s 的计算在至多 $O(p|s|)$ 步内终止, 则称 A 有多项式运行时间。
- 根据这一概念, 就形成了问题类: 存在多项式时间解法的问题的集合**P**



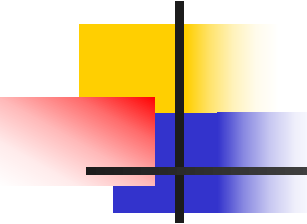
NP的定义

- PRIMES: $X = \{ 2, 3, 5, 7, 11, 13, 17, 23, 29, 31, 37, \dots \}$
- 判定素性问题:
- [Agrawal-Kayal-Saxena, 2002] AKS算法
- Manindra Agrawal, Neeraj Kayal, and Nitin Saxena on August 6, 2002 : *PRIMES is in P*.
- The authors received the 2006 Gödel Prize and the 2006 Fulkerson Prize for this work.
- $p(|s|) = |s|^{6+\epsilon}$

NP的定义

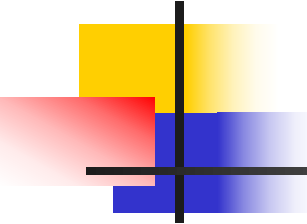
■ 一些经典的P类问题

Problem	Description	Algorithm	Yes	No
MULTIPLE	Is x a multiple of y ?	Grade school division	51, 17	51, 16
RELPRIME	Are x and y relatively prime?	Euclid (300 BCE)	34, 39	34, 51
PRIMES	Is x prime?	AKS (2002)	53	51
EDIT-DISTANCE	Is the edit distance between x and y less than 5?	Dynamic programming	niether neither	acgggt ttttta
LSOLVE	Is there a vector x that satisfies $Ax = b$?	Gauss-Edmonds elimination	$\begin{bmatrix} 0 & 1 & 1 \\ 2 & 4 & -2 \\ 0 & 3 & 15 \end{bmatrix}, \begin{bmatrix} 4 \\ 2 \\ 36 \end{bmatrix}$	$\begin{bmatrix} 1 & 0 & 0 \\ 1 & 1 & 1 \\ 0 & 1 & 1 \end{bmatrix}, \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$



NP的定义

- 直觉上告诉我们，求解问题是困难的，但是验证一个问题的解相对要容易
- 问题X的“验证算法”具有不同于求解问题的算法结构，为了验证一个解，需要输入串s以及另外的“证书”串t，这个证书t包含s是X的“yes”实例的证据。



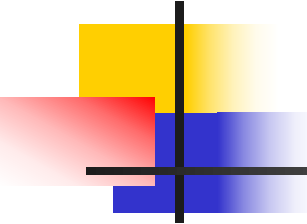
NP的定义

- 如果:
 - **B**是有两个输入变量 s, t 的多项式时间算法
 - 存在多项式 p 使得对每一个输入串 $s, s \in X$ 当且仅当存在串 t 使得 $|t| \leq p(|s|)$ 且 $B(s, t) = \text{yes}$.
- 则称**B**是问题**X**的有效验证程序。
 t 是证书



NP的定义

- 根据验证算法的性质，可以定义新的问题类别
- NP(Nondeterministic polynomial time)是所有存在有效验证程序的问题的集合



NP的定义

- 定理8.10 $P \subseteq NP$
- 证明：考虑问题 $X \in P$,意味着存在一个解 X 的多项式时间算法 A 。

如下设计 B ,对于输入 (s,t) ,验证程序直接返回值 $A(s)$.

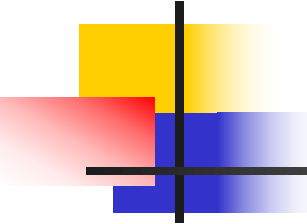
可以看到, 如果串 $s \in X$,对于每一个长度不超过 $p(|s|)$ 的 t ,满足 $B(s,t)=yes...$

可见 B 是有效验证程序。



NP的定义

- 例子：对于三元可满足性问题
- 证书 t ?
- 对所有变量的真值赋值
- 验证程序 B ?
- 计算 ϕ 在这个赋值下的值



NP的定义

$$(\bar{x}_1 \vee x_2 \vee x_3) \wedge (x_1 \vee \bar{x}_2 \vee x_3) \wedge (x_1 \vee x_2 \vee x_4) \wedge (\bar{x}_1 \vee \bar{x}_3 \vee \bar{x}_4)$$

实例 s

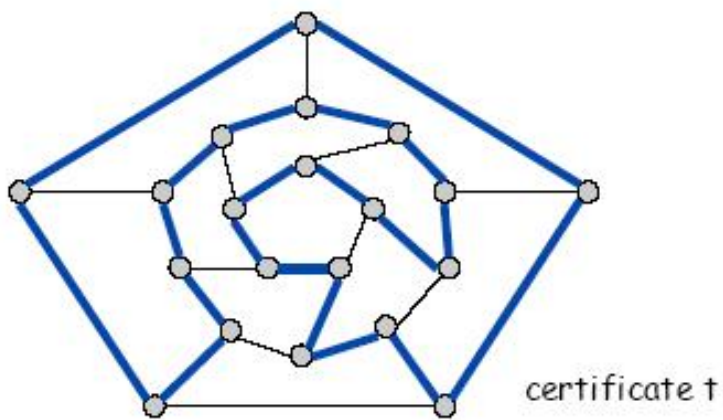
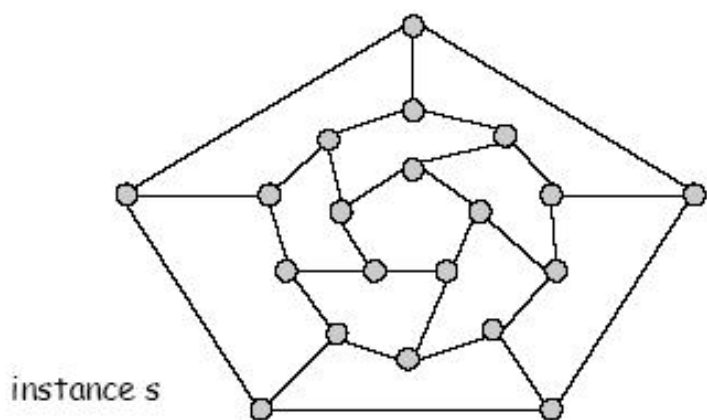
$$x_1=1, x_2=1, x_3=0, x_4=1$$

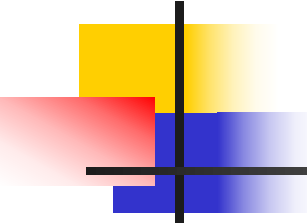
证书 t

$SAT, 3SAT \in NP$

NP的定义

- 哈密尔顿回路. 给定无向图 $G = (V, E)$, 是否存在一个简单回路 C 访问到每一个结点?
- 证书: n 个结点的一个有序排列
- 验证程序. 检查排列中包含每一个结点; 排列中相邻结点有边相连。
- $\text{HAM-CYCLE} \in \text{NP}$.



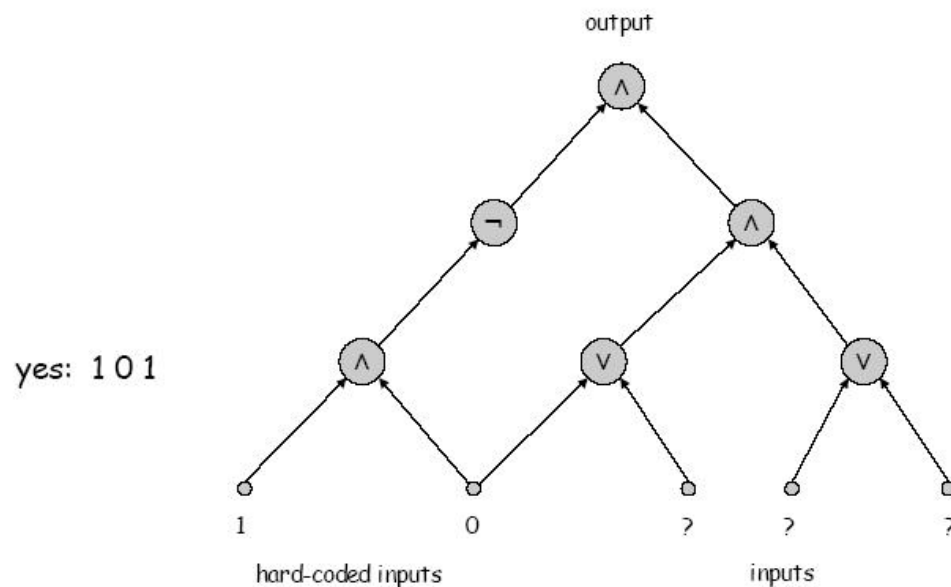


8.4 NP完全问题

- P=NP这个问题没有太多进展，转向另外一个问题，那些是NP中最难的问题？
- 自然的，要求NP中的每一个问题都能够归约到X
- NP完全问题X
 - i. $X \in \text{NP}$
 - ii. 对于所有的 $Y \in \text{NP}$, $Y \leq_p X$.

NP完全问题

- 是否存在一个天然的NP完全问题?
- 电路可满足性(CIRCUIT-SAT). 给定一个由AND, OR, NOT电路门组成的电路, 需要确定是否存在对输入的赋值使得输出值为1?

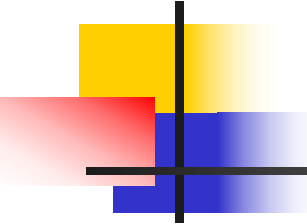




NP完全问题(电路可满足性)

定理8.13([Cook 1971, Levin 1973]) 电路可满足性是**NP**完全的

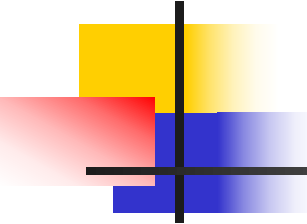
- Pf. Any algorithm that takes a fixed number of bits n as input and produces a yes/no answer can be represented by such a circuit.
Moreover, if algorithm takes poly-time, then circuit is of poly-size.
 - Consider some problem X in NP. It has a poly-time certifier $C(s, t)$.
To determine whether s is in X , need to know if there exists a certificate t of length $p(|s|)$ such that $C(s, t) = \text{yes}$.
 - View $C(s, t)$ as an algorithm on $|s| + p(|s|)$ bits (input s , certificate t) and convert it into a poly-size circuit K .
 - first $|s|$ bits are hard-coded with s
 - remaining $p(|s|)$ bits represent bits of t
 - Circuit K is satisfiable iff $C(s, t) = \text{yes}$.



NP-Complete Problems

$3\text{-SAT} \leq_p \text{独立集} \leq_p \text{顶点覆盖} \leq_p \text{集合覆盖}$

因此他们都是**NP**完全问题



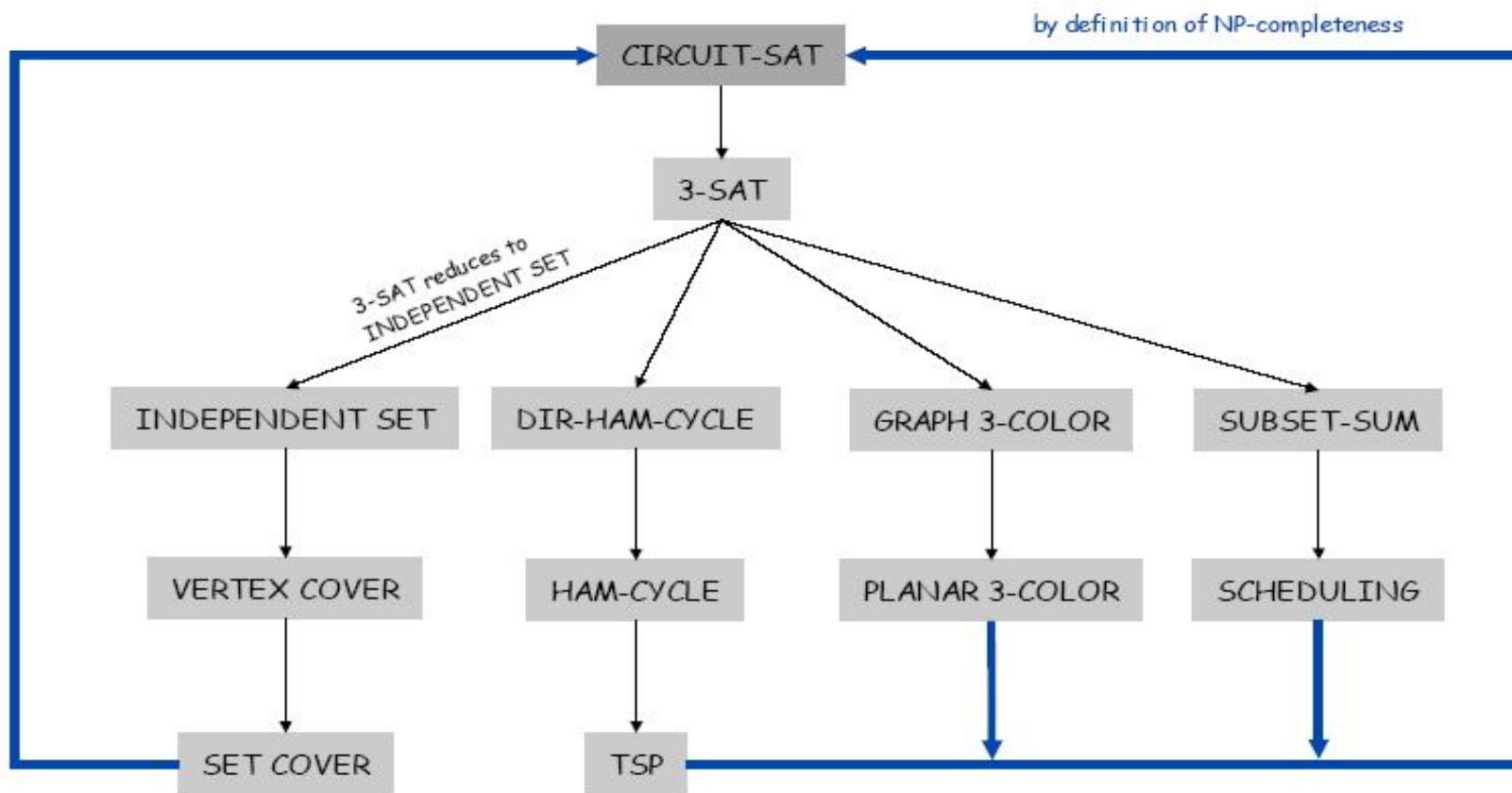
NP-Complete Problems

证明NP完全性的通用思路

1. 证明 $X \in \text{NP}$
2. 选择一个已知的NP完全问题Y
3. 考虑问题Y的任意一个实例 S_Y , 说明如何在多项式时间内构造X的一个实例 S_X :
 S_X 和 S_Y 有相同的答案。

NP完全问题

- 根据前面的多项式归约性质，我们可以得到更多的NP完全问题





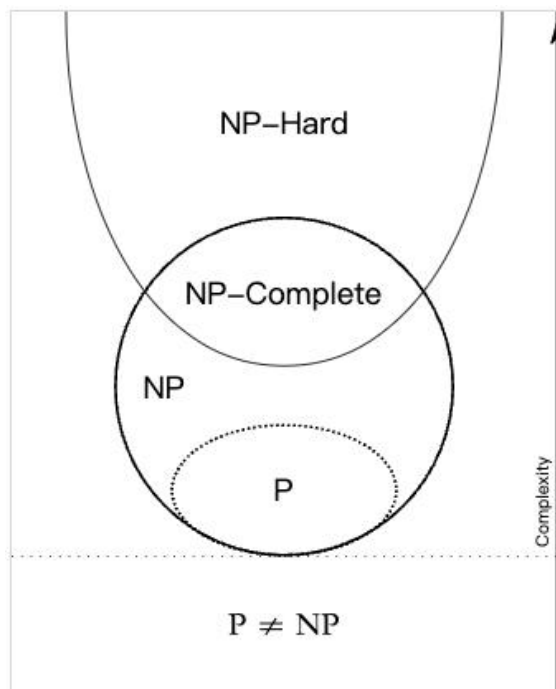
NP完全问题

- 难问题的部分分类
 1. 包装问题: 独立集, 集合包装
 2. 覆盖问题: 顶点覆盖, 集合覆盖
 3. 划分问题: 三维匹配, 图着色
 4. **排序问题**: 哈密尔顿圈, 哈密尔顿回路, 巡回售货员问题
 5. 数值问题: 子集和, 带开放时间和截止时间的调度问题
 6. 约束满足问题: **SAT, 3SAT**

- 一般的经验是, NP问题要么是P, 要么是NP完全问题

P & NP & NPC & NPH

- <http://www.matrix67.com/blog/archives/105>
- <https://yzhang-gh.github.io/notes/others/p-np.html#np-%E5%AE%8C%E5%85%A8%E9%97%AE%E9%A2%98>



- **NP-Hard问题:** 不要求是否能在多项式时间里验证一个解的问题，所有的NP问题可以（在多项式时间内）规约到它
- **NP-Complete问题:** 可以在多项式时间里验证一个解的问题，且所有的NP问题可以（在多项式时间内）规约到它
- **NP 问题:** 可以在多项式时间里验证一个解的问题
- **P 问题:** 可以在多项式时间找出解的问题